

EVALUACIÓN	Obligatorio	GRUPO	FECHA
MATERIA	Sistemas Operativos		
CARRERA	Ingeniería de Sistemas		
CONDICIONES	- Puntaje máximo: 35 puntos - Puntaje mínimo: 1 puntos - Fecha de entrega: 29/06/26 hasta las 21:00 horas en gestion.ort.edu.uy (max. 40Mb en formato zip, rar o pdf) Uso de material de apoyo y/o consulta <u>Inteligencia Artificial Generativa</u> - Seguir las pautas de los docentes: Se deben seguir las instrucciones específicas de los docentes sobre cómo utilizar la IA en cada curso. - Citar correctamente las fuentes y usos de IA: Siempre que se utilice una herramienta de IA para generar contenido, se debe citar adecuadamente la fuente y la forma en que se utilizó. - Se debe indicar: a. la /las herramientas utilizadas y b. el contexto de uso: por ejemplo, generación de ideas, redacción inicial, análisis de datos, corrección de estilo, etc. - Todo contenido producido por IAG debe ser revisado y verificado; cualquier error presente será responsabilidad del estudiante. - Ser responsables con el uso de la IA: Conocer los riesgos y desafíos, como la creación de “alucinaciones”, los peligros para la privacidad, las cuestiones de propiedad intelectual, los sesgos inherentes y la producción de contenido falso. - El uso de herramientas de Inteligencia Artificial Generativa (IAG) puede emplearse como apoyo en el proceso de aprendizaje. Sin embargo, no sustituye el razonamiento crítico ni la elaboración personal. El estudiante es responsable de que el trabajo presentado refleje su propia comprensión, análisis y proceso cognitivo sobre el tema. - La defensa es obligatoria y eliminatoria. - La defensa será presencial salvo en casos en que se habilite la defensa oral sincrónica en forma remota. Defensa Fecha de defensa: A acordar con el docente, previo al 09/07/26 <u>La defensa es obligatoria y eliminatoria.</u> El docente es quien definirá y comunicará la modalidad, y mecánica de defensa. <u>La no presentación a la misma implica la pérdida de la totalidad de los puntos del Obligatorio.</u> IMPORTANTE: 1) Inscribirse 2) Formar grupos de hasta 3 personas del mismo dictado 3) Subir el trabajo a Gestión antes de la hora indicada (ver hoja al final del documento: “RECORDATORIO”)		

Aquellos de ustedes que <u>presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos</u> , por favor contactarse con la Coordinadora de cursos o Coordinación Adjunta antes de las 20:00h del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy .

Parte 1 - Bash

Gestión de Inventario "El Paddock"

Estás trabajando como SysAdmin para el equipo de logística del Gran Prix. El *Team Principal* utiliza un script de Bash heredado (*paddock_manager.sh*) para gestionar el inventario de mercadería del fin de semana (gorras, remeras, camperas).

El sistema almacena los datos en un archivo llamado *inventario_f1.csv* con el formato:
ID,Nombre_del_Producto,Escuderia,Stock,Precio

Además, existe un directorio llamado */mercaderia/* donde se guardan manifiestos de envío en texto plano por cada escudería (ej. *Ferrari_manifiesto.txt*).

Durante la última carrera en Monza, el sistema reportó fallos críticos que casi dejan al equipo sin control de stock. Los usuarios reportaron los siguientes tickets de error:

1. **Ticket #104 (Fallo en Consultas):** *"Cuando busco productos simples como 'Gorra' el sistema funciona. Pero cuando intento buscar artículos específicos como 'Gorra Edicion Especial Monza', la terminal me devuelve un error diciendo que el archivo 'Edicion' no existe. El inventario claramente no se está leyendo bien."*
2. **Ticket #105 (Pérdida de Datos Catastrófica):** *"Un pasante intentó usar la función de descatalogar una escudería, pero olvidó ingresar el nombre de la misma al ejecutar el script. El comando se ejecutó igual y, misteriosamente, se borraron TODOS los manifiestos de todas las escuderías en la carpeta /mercaderia/."*

Objetivo:

No se debe reescribir el sistema desde cero. Se debe investigar el código del script provisto y realizar las siguientes tareas:

1. **Informe de Autopsia (Diagnóstico):** Explica técnicamente por qué ocurrieron los incidentes #104 y #105. Debes identificar la línea exacta del fallo y explicar qué hizo el intérprete de Bash (conceptos de Word Splitting, Globbing y expansión de variables).
2. **El Parche:** Entrega el script modificado (*paddock_manager_fixed.sh*) aplicando buenas prácticas de scripting (quoting, validación de variables vacías y manejo seguro de rutas) para que sea robusto ante errores humanos.

```
#!/bin/bash

# paddock_manager.sh - V1.0 (Legacy)
# Uso: ./paddock_manager.sh [accion] [parametros...]

ACCION=$1
PARAMETRO=$2

ARCHIVO_CSV="inventario_f1.csv"
DIR_MERCADERIA="./mercaderia"

if [ -z "$ACCION" ]; then
    echo "Error: Debes especificar una accion (ingresar, buscar, vender,
descatalogar)."
```

```
    exit 1
fi

case $ACCION in
    buscar)
        echo "Buscando '$PARAMETRO' en el inventario..."
        grep $PARAMETRO $ARCHIVO_CSV
        ;;

    descatalogar)
        echo "Descatalogando productos y manifiestos de la escuderia:
$PARAMETRO"
        rm $DIR_MERCADERIA/$PARAMETRO*.txt

        # Borrado del CSV
        sed -i "/$PARAMETRO/d" $ARCHIVO_CSV
        echo "Operacion finalizada."
        ;;

    ingresar)
        echo "Ingresando nuevo producto..."
        echo $PARAMETRO >> $ARCHIVO_CSV
        echo "Producto ingresado."
        ;;
```

```
vender)
    echo "Vendiendo 1 unidad del ID: $PARAMETRO"
    echo "Función en mantenimiento..."
    ;;

*)
    echo "Acción no reconocida."
    exit 1
    ;;
esac
```

Parte 2: Resolución en C

Ejercicio 1:



En un restaurante muy concurrido del centro, la cocina y el salón trabajan de forma coordinada para atender a los clientes de la manera más eficiente posible.

Detrás de escena, los cocineros preparan platos constantemente, mientras que los mozos se encargan de retirarlos y llevarlos a las mesas. Entre ambos existe un elemento clave: una barra de platos listos, donde los cocineros dejan los platos terminados y los mozos los retiran. Sin embargo, el

espacio en esta barra es limitado, y la coordinación entre ambos roles es fundamental para evitar problemas en el servicio. Si la barra se llena, los cocineros deben esperar antes de colocar nuevos platos. Si la barra está vacía, los mozos deberán esperar hasta que haya platos disponibles. También sucede que la barra es estrecha y entra un solo mozo a la vez.

El problema consiste en diseñar un mecanismo que permita coordinar correctamente el trabajo de cocineros y mozos, asegurando que no haya inconsistencias en el uso de la barra.

Requerimientos

- El sistema debe considerar:
 - 5 cocineros
 - 10 mozos
- Existe una barra de platos listos con capacidad máxima de 8 platos
- Cada cocinero:
 - Prepara un plato en un tiempo random no mayor a 3 segundos
 - Luego intenta colocarlo en la barra
- Cada mozo:
 - Retira un plato de la barra
 - Lo entrega en un tiempo random no mayor a 2 segundos
- El sistema debe finalizar cuando se hayan preparado y entregado 50 platos en total
- Debe garantizarse la correcta sincronización entre cocineros y mozos

Ejemplo de salida :

Cocinero 2 preparó un plato

Cocinero 2 dejó un plato en la barra (platos en barra: 5)

Mozo 4 retiró un plato de la barra (platos en barra: 4)

Mozo 4 está entregando el plato

Notas

- Se debe utilizar **threads y semáforos como fueron vistos en clase**
- Se valorará que la solución mantenga una estructura clara y ordenada
- El alumno deberá ser capaz de explicar el funcionamiento de la sincronización implementada

Ejercicio 2:

Un equipo de desarrollo debe desplegar una aplicación siguiendo un conjunto de tareas con dependencias.

Las tareas y sus precedencias son las siguientes:

- Inicializar sistema (IS): Sin previas

- Configurar base de datos (BD): Requiere IS
- Configurar servidor (SV): Requiere IS
- Ejecutar pruebas (PR): Requiere BD y SV
- Deploy final (DF): Requiere PR

Se pide

1. Dibujar el grafo de precedencias de las tareas.
2. Explicar cómo se inicializarían los semáforos para garantizar el cumplimiento de las dependencias.
3. Justificar brevemente por qué dicha inicialización es correcta.

Nota

- El alumno deberá ser capaz de **explicar claramente en la defensa** el grafo realizado y la lógica de inicialización de los semáforos utilizados.
- En caso de no poder justificar correctamente la solución, podrá considerarse como no satisfactorio.

Parte 3. ADA.

Se debe hacer un space invaders simplificado.

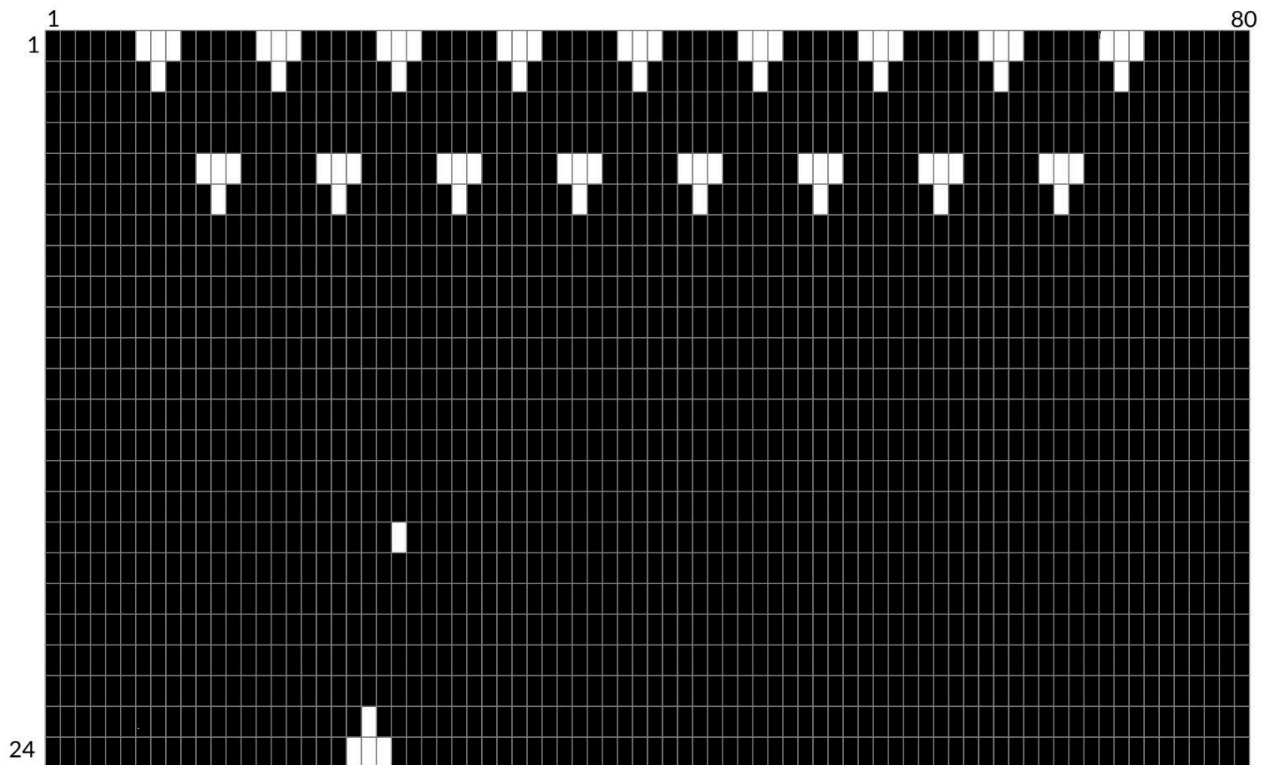
El juego consta de una pantalla de 80 caracteres de ancho por 24 de alto (la pantalla estándar de consola, puede variar ligeramente estas medidas si lo desea). En esa pantalla hay una cantidad a determinar de naves espaciales que son objetos de 4 píxeles. Y que están alternadas en al menos dos filas de naves. Estas naves se mueven lentamente hacia la derecha primero y hacia la izquierda después.

También existe un cañón espacial, el cual controlamos, sobre la última fila de la pantalla (Debajo). Tiene la forma de las naves espaciales pero mirando hacia arriba.

Este cañón es controlado por la consola, un carácter ('D') lo envía un lugar a la derecha y otro ('A') un lugar a la izquierda, y ('W') le hace disparar una bala (puede cambiar las letras si lo desea). El cañón siempre está contra el fondo (debajo) de la pantalla. Sólo se mueve a izquierda y derecha (hasta el límite de la pantalla). No hacia arriba o abajo. Cada bala se crea en la punta del cañón y se dirige paso a paso en forma vertical hacia arriba. La bala se mueve en un solo eje (vertical). Podemos disparar otras balas mientras las balas anteriores aún estén viajando a su objetivo o errando y desapareciendo en la primera línea de la pantalla. Mientras la bala viaja, es posible que el cañón y las naves se muevan. Puede considerar que una nave se destruye caracter a caracter o destruir la nave por completo cuando uno de sus caracteres es alcanzado por la bala. El juego termina al terminar las naves enemigas. El juego funciona en una consola de texto.

Consideraciones:

- Cada nave es una tarea.
- La bala también lo es.
- Deberá definir la colisión como una cita.
- Existirán iteraciones donde no hay citas entre tareas.
- En cada iteración se deberá leer la entrada del usuario y actuar en consecuencia (redibujar)



La forma de mostrar las naves, cañón, bala y espacio, puede variar en caso de que lo desee y utilizar otros caracteres.

Parte 4 - Virtualización

Empaquetando apps

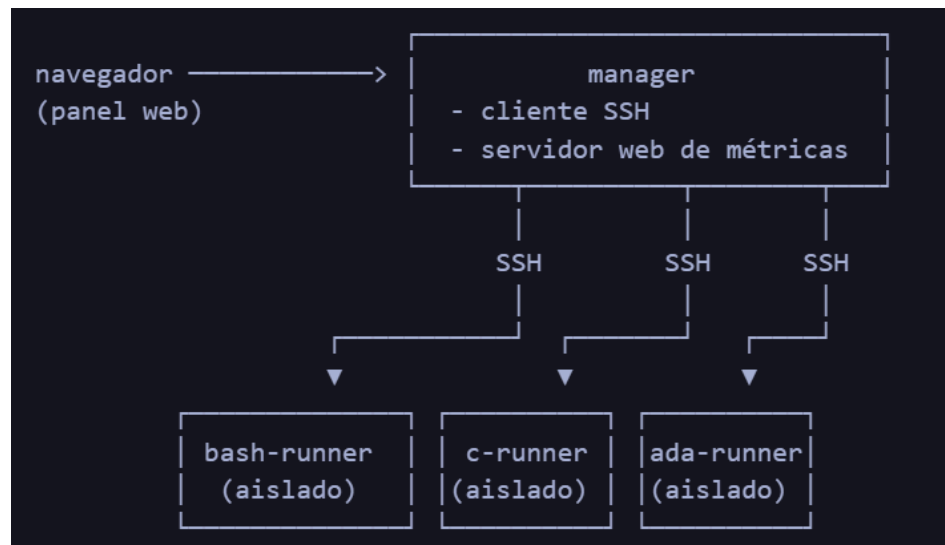
En las partes 1 a 3 desarrollaron programas que trabajan con conceptos fundamentales de sistemas operativos: scripting en Bash, programación concurrente en C con hilos POSIX, y concurrencia estructurada en ADA con tasks.

El objetivo es empaquetar cada aplicación en su propio contenedor Docker aislado, cumpliendo criterios estrictos de mínima superficie y mínima vulnerabilidad, y coordinar todo desde un contenedor gestor que permite acceder a cada entorno vía SSH y visualizar el estado del sistema desde una interfaz web.

No se provee ningún código base ni scaffolding. El diseño, la implementación y las justificaciones son completamente responsabilidad del grupo.

Arquitectura solicitada

El sistema debe estar compuesto por exactamente cuatro contenedores, orquestados con Docker Compose:



Contenedores de ejecución (runners)

Cada runner encapsula una única aplicación desarrollada en la parte correspondiente del obligatorio. Con las siguientes características:

- Contener únicamente lo estrictamente necesario para ejecutar esa aplicación. No más.
- Exponer un servidor SSH para que el manager pueda conectarse y ejecutar la aplicación remotamente.

- No exponer ningún puerto al host — solo son accesibles desde la red interna Docker.
- Operar con la mínima cantidad de vulnerabilidades posible, tanto a nivel de imagen base como de configuración.

Contenedor manager

El manager tiene dos responsabilidades:

1. Acceso SSH a los runners

El manager es el único punto de entrada al sistema. Desde él, un operador puede conectarse por SSH a cualquiera de los runners para ejecutar las aplicaciones.

2. Panel web de monitoreo

El manager debe servir una interfaz web accesible en el host (localhost:8080 o el puerto que elijan) que muestre, como mínimo:

- Estado de cada runner (activo / inactivo).
- Métricas básicas de cada contenedor: uso de CPU, memoria, y cualquier otra que consideren relevante.

La tecnología para implementar el panel web queda a criterio del grupo.

Requisitos

Funcionales

- Las tres aplicaciones (Bash, C, ADA) deben poder ejecutarse correctamente desde el manager vía SSH hacia el runner correspondiente.
- El panel web del manager debe estar operativo y mostrar información real de los contenedores.
- El stack completo debe levantarse con un único comando: docker compose up.

De seguridad y eficiencia

Requisito	Descripción
Imagen base mínima	La imagen base de cada contenedor debe ser la más liviana posible que permita cumplir el rol. La elección debe estar fundamentada.

Requisito	Descripción
Usuario no root	El proceso principal de cada contenedor no puede correr como root.
Sistema de archivos de solo lectura	El filesystem del contenedor debe ser de solo lectura donde sea posible.
Capabilities mínimas	Se deben eliminar todas las Linux capabilities innecesarias. Solo se agregan las que el contenedor realmente requiere.
Sin escalada de privilegios	Se debe prevenir la escalada de privilegios dentro del contenedor.
Puertos mínimos	Ningún runner expone puertos al host. El manager expone solo los necesarios.
SSH endurecido	La configuración del servidor SSH en los runners debe seguir buenas prácticas de seguridad (deshabilitar login por contraseña, deshabilitar root login, etc.).
Sin software innecesario	Ningún contenedor debe incluir herramientas, intérpretes o librerías que no use.

Se debe entregar:

- **Código fuente**
- **Informe técnico:** Debe demostrar que el grupo tomó decisiones informadas y comprende las implicaciones de cada una. También debe incluir instrucciones claras de cómo levantar la solución y cómo conectarse vía SSH a cada runner desde el manager
 - Se recomienda cubrir los siguientes puntos para cada contenedor:
 - **Elección de imagen base**
 - **Análisis de seguridad** (con evidencia del análisis, resumen de vulnerabilidades encontradas, etc.)
 - **Configuración de seguridad aplicada**
 - **Decisiones de diseño adicionales**

Rubrica

35 puntos total

- Ejercicio Bash (5pts)
 - El alumno es capaz de describir el funcionamiento de la totalidad de los comandos utilizados y sus parámetros. (4 pts)
 - El programa hace lo pedido en su totalidad (1 pts)
- Ejercicio POSIX (7 pts)
 - El alumno es capaz de describir el funcionamiento de la totalidad de los comandos utilizados y sus parámetros. (5 pts)
 - El programa hace lo pedido en su totalidad (2 pts)
- Ejercicio ADA (16 pts)
 - El alumno conoce el funcionamiento de cada sección de código, conociendo cada sentencia que funcionalidad realiza. En caso de presentar sentencias de código no dadas en clase o ajenas a los laboratorios, el alumno debe mostrar total soltura en su manejo de las mismas. (10 Pts)
 - 2 el ejercicio es resuelto usando analogías a problemas vistos en clase. Manteniéndose simple en su solución. (2 Pts)
 - El programa hace lo pedido en su totalidad (4 pts)
- Ejercicio Docker (7 pts)
 - El alumno es capaz de describir el funcionamiento de la totalidad de los comandos utilizados y sus parámetros. (4 pts)
 - El programa hace lo pedido en su totalidad (1 pts)

Nota: Esta rúbrica representa una guía a la hora de distribuir los puntos. El docente puede definir bajo su criterio el valor en puntaje del no cumplimiento de alguno de los puntos.

Errores no expresados en la rúbrica y casuísticas que puedan surgir pueden ser sopesados y tenidos en cuenta para la evaluación por el docente (como ser la documentación).

Todas las defensas son presenciales y obligatorias, las mismas pueden ser grupales o individuales según considere el docente, las notas son individuales.

Existe un foro para anunciar criterios a la letra tomados en situaciones particulares que surjan.

RECORDATORIO: IMPORTANTE PARA LA ENTREGA

La entrega de los obligatorios será en formato digital online, a excepción de algunas materias que se entregarán en Bedelía y en ese caso recibirá información específica en el dictado de la misma.

Los principales aspectos a destacar sobre la **entrega online de obligatorios** son:

1. Ingresá al sistema de Gestión.
2. En el menú, seleccioná el ítem “Evaluaciones” y la instancia de evaluación correspondiente, que figura bajo el título “Inscripto”.
3. Para iniciar la entrega hacé clic en el ícono: 
4. Ingresá el número de estudiante de cada uno de los integrantes y hacé clic en “Agregar”. El sistema confirmará que los integrantes estén inscriptos al obligatorio y, de ser así, mostrará el nombre y la fotografía de cada uno de ellos. Una vez agregados todos los integrantes, hacé clic en “Crear equipo”.

Cualquier integrante podrá:

- **Modificar la integración del equipo.**
- **Subir el archivo de la entrega.**

5. Seleccioná el archivo que deseás entregar. Verificá el nombre del archivo que aparecerá en la pantalla y hacé clic en “Subir” para iniciar la entrega. Cada equipo (hasta 2 estudiantes) debe entregar **un único archivo en formato zip o rar** (los documentos de texto deben ser pdf, y deben ir dentro del zip o rar). El archivo a subir debe tener **un tamaño máximo de 40mb**

Cuando el archivo quede subido, se mostrará el nombre generado por el sistema (1), el tamaño y la fecha en que fue subido.

6. El sistema enviará un e-mail a todos los integrantes del equipo informando los detalles del archivo entregado y confirmando que la entrega fue realizada correctamente.
7. Podés cerrar la pestaña de entrega y continuar utilizando Gestión o salir del sistema.
8. La **hora tope para subir el archivo será las 21:00** del día fijado para la entrega.
9. La entrega se podrá realizar desde cualquier lugar (ej. hogar del estudiante, laboratorios de la Universidad, etc).
10. Aquellos de ustedes que presenten alguna dificultad con su inscripción o tengan inconvenientes técnicos, por favor contactarse con la Coordinadora de Cursos o Coordinación Adjunta **antes de las 20:00h** del día de la entrega, a través del mail: adjuntos_ei@ort.edu.uy.